

# **JOB PORTAL MANAGEMENT SYSTEM**

*School of Computing*

**Bachelor of Technology  
in  
Computer Science & Engineering**

**By**

**POSHAM LIKITHA HARSHINI DEVI(VTU26654)**

*10211CS224*

*Full Stack Application Development*

*Slot : E1*



**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING  
SCHOOL OF COMPUTING**

**VEL TECH RANGARAJAN Dr.SAGUNTHALA R&D  
INSTITUTE OF SCIENCE AND TECHNOLOGY  
(Deemed to be University Estd u/s 3 of UGC Act, 1956)  
CHENNAI 600 062, TAMILNADU, INDIA**

**May, 2026**

# **BONAFIDE CERTIFICATE**

It is certified that the work contained in the Full Stack Application Development report titled "JOB PORTAL MANAGEMENT SYSTEM" by P LIKITHA HARSHINI DEVI(VTU26654) has been carried out in Winter semester of Academic year 2025-2026(WS2526).

**Signature of Faculty**

**Dr. Hemamalini.S**

**Assistant Professor(Senior Grade)**

**Computer Science & Engineering**

**School of Computing**

**Vel Tech Rangarajan Dr.Sagunthala R&D**

**Institute of Science and Technology**

**May, 2026**

**Signature of Course Coordinator**

**Dr. Sumithra.M**

**Assistant Professor(Senior Grade)**

**Computer Science & Engineering**

**School of Computing**

**Vel Tech Rangarajan Dr.Sagunthala R&D**

**Institute of Science and Technology**

**May, 2026**

# **DECLARATION**

I declare that this written submission represents my ideas in my own words and where others' ideas or words have been included, we have adequately cited and referenced the original sources. I also declare that i have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in our submission. I understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

(P LIKITHA HARSHIINI DEVI)

Date: 06/05/2026

# ABSTRACT

This project presents the design and development of a web-based Job Portal Application using Spring Boot, Thymeleaf, and MySQL. The system provides a platform for job seekers to search and apply for jobs, and for employers to post job openings and manage applications.

The application follows the Model-View-Controller (MVC) architecture, ensuring proper separation between the user interface, business logic, and data management. Spring Boot is used for backend development, Thymeleaf for dynamic web pages, and MySQL for database storage.

The system includes features such as user registration, login authentication, job posting, and application management. It improves the efficiency of recruitment processes and reduces manual effort.

Overall, the project demonstrates the effective use of modern web technologies in building a simple and efficient job portal system.

**Keywords :** Job Portal, Spring Boot, Thymeleaf, MySQL, MVC Architecture, Web Application

# LIST OF FIGURES

|     |                                                         |    |
|-----|---------------------------------------------------------|----|
| 1.1 | System Architecture of Job Portal Application . . . . . | 4  |
| 3.1 | System Flow Diagram . . . . .                           | 11 |
| 4.1 | MVC Architecture of Job Portal System . . . . .         | 14 |
| 4.2 | Data Flow Diagram . . . . .                             | 15 |
| 5.1 | Login Output Screen . . . . .                           | 20 |
| 5.2 | Employer Dashboard Output Screen . . . . .              | 20 |
| 5.3 | Edit Job Output Screen . . . . .                        | 21 |
| 5.4 | Student Dashboard Output Screen . . . . .               | 21 |
| 5.5 | Student Profile Output Screen . . . . .                 | 22 |

# LIST OF TABLES

|     |                                                              |    |
|-----|--------------------------------------------------------------|----|
| 5.1 | Comparison of Existing Systems and Proposed System . . . . . | 19 |
| 7.1 | Mapping of CEP Attributes . . . . .                          | 31 |
| 7.2 | Mapping of CEA Attributes . . . . .                          | 32 |
| 7.3 | SDG Mapping . . . . .                                        | 32 |

# **LIST OF ACRONYMS AND ABBREVIATIONS**

|      |                                   |
|------|-----------------------------------|
| MVC  | Model View Controller             |
| API  | Application Programming Interface |
| JPA  | Java Persistence API              |
| JDBC | Java Database Connectivity        |
| SQL  | Structured Query Language         |
| HTML | HyperText Markup Language         |
| CSS  | Cascading Style Sheets            |
| JS   | JavaScript                        |
| HTTP | HyperText Transfer Protocol       |
| DBMS | Database Management System        |

# TABLE OF CONTENTS

|                                                  | Page.No    |
|--------------------------------------------------|------------|
| <b>ABSTRACT</b>                                  | <b>iii</b> |
| <b>LIST OF FIGURES</b>                           | <b>iv</b>  |
| <b>LIST OF TABLES</b>                            | <b>v</b>   |
| <b>LIST OF ACRONYMS AND ABBREVIATIONS</b>        | <b>vi</b>  |
| <b>1 INTRODUCTION</b>                            | <b>1</b>   |
| 1.1 Introduction . . . . .                       | 1          |
| 1.2 Aim of the Project . . . . .                 | 2          |
| 1.3 Scope of the Project . . . . .               | 2          |
| 1.4 Framework . . . . .                          | 3          |
| <b>2 SURVEY OF SIMILAR APPLICATION IN MARKET</b> | <b>5</b>   |
| 2.1 LinkedIn . . . . .                           | 5          |
| 2.2 Naukri . . . . .                             | 6          |
| 2.3 Indeed . . . . .                             | 6          |
| 2.4 Comparison with Proposed System . . . . .    | 7          |
| <b>3 FRAMEWORK DESCRIPTION</b>                   | <b>8</b>   |
| 3.1 Frontend Implementation . . . . .            | 8          |

|          |                                                                  |           |
|----------|------------------------------------------------------------------|-----------|
| 3.2      | Backend Implementation . . . . .                                 | 9         |
| 3.3      | Database Implementation . . . . .                                | 12        |
| <b>4</b> | <b>METHODOLOGIES</b>                                             | <b>13</b> |
| 4.1      | Project Architecture . . . . .                                   | 13        |
| 4.2      | DataFlow Diagram . . . . .                                       | 15        |
| 4.3      | Module 1: User Management . . . . .                              | 16        |
| 4.4      | Module 2: Job Management . . . . .                               | 16        |
| 4.5      | Module 3: Application Management . . . . .                       | 17        |
| <b>5</b> | <b>RESULTS AND DISCUSSIONS</b>                                   | <b>18</b> |
| 5.1      | System Screenshots and Results . . . . .                         | 20        |
| <b>6</b> | <b>CONCLUSION AND FUTURE ENHANCEMENTS</b>                        | <b>23</b> |
| 6.1      | Conclusion . . . . .                                             | 23        |
| 6.2      | Future Enhancements . . . . .                                    | 24        |
| 6.3      | Existing Applications and Review . . . . .                       | 24        |
| 6.3.1    | Application Access . . . . .                                     | 25        |
| 6.4      | SOURCE CODE IMPLEMENTATION . . . . .                             | 26        |
| <b>7</b> | <b>ADDRESSING COMPLEX ENGINEERING PROBLEM AND SDG</b>            | <b>31</b> |
| 7.1      | Mapping of the Project to Complex Engineering Problem (CEP) . .  | 31        |
| 7.2      | Mapping of the Project to Complex Engineering Activities (CEA) . | 32        |
| 7.3      | SDG Mapping . . . . .                                            | 32        |
|          | <b>References</b>                                                | <b>33</b> |

# Chapter 1

## INTRODUCTION

### 1.1 Introduction

The rapid growth of the internet has significantly transformed the way recruitment and job searching are carried out. Traditional methods of job application and hiring are often time-consuming, inefficient, and lack proper organization. To overcome these challenges, a web-based Job Portal Application has been developed using modern technologies such as Spring Boot, Thymeleaf, and MySQL.

This application provides a centralized platform where job seekers can search and apply for jobs, while employers can post job vacancies and manage applications effectively. The system ensures seamless communication between users and improves the overall recruitment process. By automating various tasks, the application reduces manual effort and enhances efficiency.

The project follows the Model-View-Controller (MVC) architecture, which ensures proper separation of concerns between different layers of the application. The frontend is designed using Thymeleaf templates, the backend logic is implemented using Spring Boot, and MySQL is used for storing and managing data.

## **1.2 Aim of the Project**

The main aim of this project is to design and develop a user-friendly and efficient Job Portal system that simplifies the recruitment process for both job seekers and employers.

The objectives of the project include:

- To provide a platform for users to register and log in securely.
- To enable job seekers to search and apply for jobs.
- To allow employers to post job vacancies and manage applicants.
- To maintain user and job data efficiently using a database system.
- To implement a scalable and maintainable web application using Spring Boot.

## **1.3 Scope of the Project**

The scope of this project includes the development of a web-based application that facilitates job searching and recruitment activities. The system is designed to handle multiple users simultaneously and provide a responsive and interactive user experience.

The application covers:

- User registration and authentication.
- Job posting and management by employers.
- Job search and application by job seekers.
- Database management using MySQL.
- Basic security features for user data protection.

However, the system currently focuses on core functionalities and can be extended in the future with advanced features such as email notifications, resume uploads, and AI-based job recommendations.

## 1.4 Framework

The project is developed using the Spring Boot framework, which simplifies the development of Java-based web applications. It follows the MVC (Model-View-Controller) architecture:

- **Model:** Represents the data layer, including entities and database interactions.
- **View:** Developed using Thymeleaf templates to provide a dynamic user interface.
- **Controller:** Handles user requests and controls the application flow.

Spring Boot provides features such as dependency injection, embedded servers, and easy integration with databases, making it suitable for building scalable and efficient applications.

The overall system architecture of the Job Portal Application is shown in [Figure 1.1]. It illustrates the interaction between the user interface, controller, and database layers, ensuring efficient data flow within the system.

## SYSTEM ARCHITECTURE OF JOB PORTAL APPLICATION

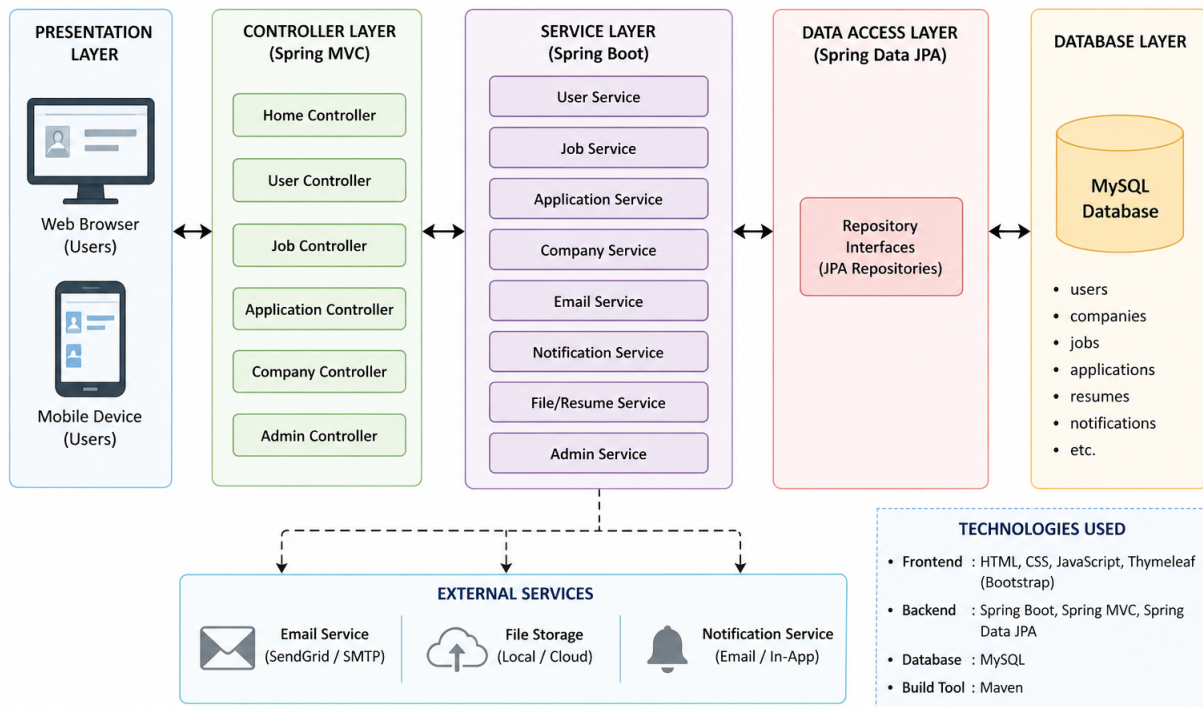


Figure 1.1: System Architecture of Job Portal Application

Figure 1.1 shows the system architecture of the Job Portal application, illustrating the layered structure of the system. The presentation layer consists of the user interface accessed through web browsers and mobile devices. The controller layer, implemented using Spring MVC, handles user requests and directs them to appropriate services. The service layer, built with Spring Boot, contains the business logic such as user management, job services, application handling, and notifications. The data access layer uses Spring Data JPA repositories to interact with the database efficiently. Finally, the database layer stores all relevant data including users, jobs, applications, and roles in MySQL. External services such as email notifications and file storage are also integrated to enhance system functionality. This architecture ensures scalability, modularity, and efficient data flow within the application.

# **Chapter 2**

## **SURVEY OF SIMILAR APPLICATION IN MARKET**

The rapid advancement of web technologies has led to the development of various online job portal platforms that connect job seekers with employers. These platforms provide efficient solutions for recruitment and job searching. A study of existing systems helps in understanding their features, advantages, and limitations, which in turn assists in designing an improved application.

Some of the popular job portal applications available in the market are:

### **2.1 LinkedIn**

LinkedIn is one of the most widely used professional networking platforms. It allows users to create professional profiles, connect with industry experts, and apply for jobs. Recruiters can post job openings and search for candidates based on their skills and experience.

#### **Advantages:**

- Strong professional networking features
- Job recommendations based on user profiles

- Easy communication between recruiters and candidates

**Limitations:**

- Complex interface for new users
- Many features require premium subscription

## **2.2 Naukri**

Naukri is a popular job portal widely used in India. It provides a platform for job seekers to upload resumes and apply for jobs, while employers can search for suitable candidates.

**Advantages:**

- Large database of job listings
- Resume upload and profile management
- Simple job search functionality

**Limitations:**

- Limited personalization of job recommendations
- Frequent notifications may be distracting

## **2.3 Indeed**

Indeed is a global job search platform that aggregates job listings from various sources. It allows users to search for jobs based on location, salary, and job type.

**Advantages:**

- Wide range of job listings from multiple sources

- Easy-to-use interface
- Free access for job seekers

**Limitations:**

- Duplicate job postings may appear
- Limited direct interaction with employers

## **2.4 Comparison with Proposed System**

The proposed Job Portal Application aims to overcome some of the limitations of existing systems by providing a simple, user-friendly interface and essential functionalities without unnecessary complexity. Unlike existing platforms, the system focuses on ease of use, efficient job management, and streamlined communication between users.

**Key Improvements:**

- Simplified user interface
- Focused features for academic-level implementation
- Efficient database management using MySQL
- Secure authentication using Spring Boot

Thus, the proposed system provides a balanced solution by combining essential features of existing applications while maintaining simplicity and efficiency.

# Chapter 3

## FRAMEWORK DESCRIPTION

### 3.1 Frontend Implementation

#### 3.1.1 Environment Setup

The frontend of the application is developed using HTML, CSS, and Thymeleaf templates. Thymeleaf is a server-side Java template engine used for rendering dynamic web pages in Spring Boot applications.

The development environment includes:

- Visual Studio Code (VS Code) as the code editor
- Web browser (Chrome/Edge) for testing
- Maven for project dependency management

Thymeleaf integrates seamlessly with Spring Boot, enabling dynamic data rendering from the backend to the frontend.

#### 3.1.2 Structure of the Project

The frontend files are organized within the project as follows:

- **templates/** – Contains Thymeleaf HTML files such as login, register, dashboard, and profile pages.

- **static/** – Contains static resources like CSS, JavaScript, and images.

Each HTML page is designed to interact with backend controllers and display dynamic content to users.

### **3.1.3 Execution Procedure**

- Start the Spring Boot application.
- Open a web browser and navigate to the application URL (e.g., <http://localhost:8081>).
- The frontend pages are loaded dynamically using Thymeleaf.
- Users can interact with forms such as login, registration, and job applications.

## **3.2 Backend Implementation**

### **3.2.1 Environment Setup**

The backend is developed using Spring Boot, a Java-based framework that simplifies the creation of web applications.

The environment setup includes:

- Java Development Kit (JDK 17 or above)
- Spring Boot framework
- Maven for dependency management
- MySQL Driver (mysql-connector)

Spring Boot provides built-in support for REST APIs, dependency injection, and database connectivity.

### **3.2.2 Structure of the Project**

The backend follows the MVC (Model-View-Controller) architecture and is organized into the following packages:

- **controller/** – Handles user requests and responses.
- **service/** – Contains business logic of the application.
- **repository/** – Handles database operations using JPA.
- **entity/** – Represents database tables as Java classes.
- **config/** – Contains configuration-related classes.
- **security/** – Manages authentication and authorization.

### 3.2.3 Execution Procedure

- Run the application using Maven command: `mvn spring-boot:run`.
- Spring Boot initializes the embedded server.
- Controllers handle incoming HTTP requests.
- Services process the business logic.
- Repositories interact with the database.
- Responses are sent back to the frontend.

The working flow of the system is illustrated in [Figure3.1]. It shows how requests move from the frontend to the backend and how responses are returned after processing.

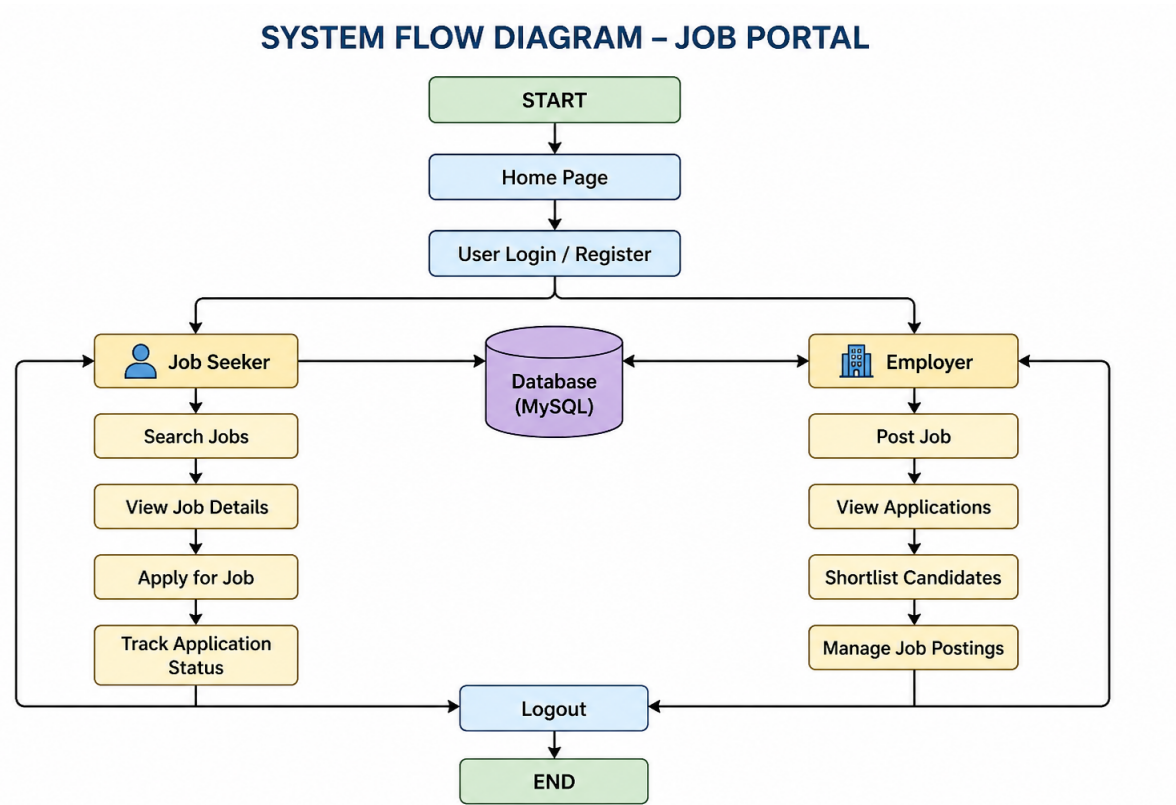


Figure 3.1: System Flow Diagram

Figure 3.1 shows the system flow diagram of the Job Portal, representing the overall working process of the application. The process begins with the home page, where users can either register or log in to the system. Based on the user role, the flow is divided into job seeker and employer functionalities. Job seekers can search for jobs, view job details, apply for jobs, and track their application status. Employers, on the other hand, can post jobs, view applications, shortlist candidates, and manage job postings. All the data is stored and retrieved from the MySQL database, ensuring efficient data management. The process concludes with the logout operation, marking the end of the session. This diagram clearly illustrates the interaction between different user roles and system components.

## 3.3 Database Implementation

### 3.3.1 Database Configuration

The database used in this project is MySQL. It is configured using the application.properties file in Spring Boot.

The configuration includes:

- Database URL (jdbc:mysql://localhost:3306/student\_db)
- Username and password
- Hibernate properties for automatic table creation and updates

Spring Boot connects to MySQL using JDBC and manages data using JPA (Java Persistence API).

### 3.3.2 Schema Structure

The database schema is designed to store user and job-related information efficiently. It includes multiple tables that represent different entities in the system.

The schema ensures:

- Proper data organization
- Efficient querying
- Relationships between entities

### 3.3.3 Tables created

The main tables created in the database include:

- **users** – Stores user details such as ID, name, email, and password.
- **jobs** – Stores job postings including title, description, and company details.
- **applications** – Stores job application details linking users and jobs.

# Chapter 4

## METHODOLOGIES

### 4.1 Project Architecture

The project follows the Model-View-Controller (MVC) architecture, which separates the application into three main components: Model, View, and Controller. This separation improves maintainability, scalability, and code organization.

- **Model:** Represents the data layer, including entity classes and database interactions using JPA.
- **View:** Implemented using Thymeleaf templates to display dynamic content to users.
- **Controller:** Handles incoming requests, processes them using services, and returns appropriate responses.

The architecture ensures a smooth flow of data between the user interface and the database. The overall structure of the system is shown in [Figure4.1], which illustrates the interaction between different layers of the application.

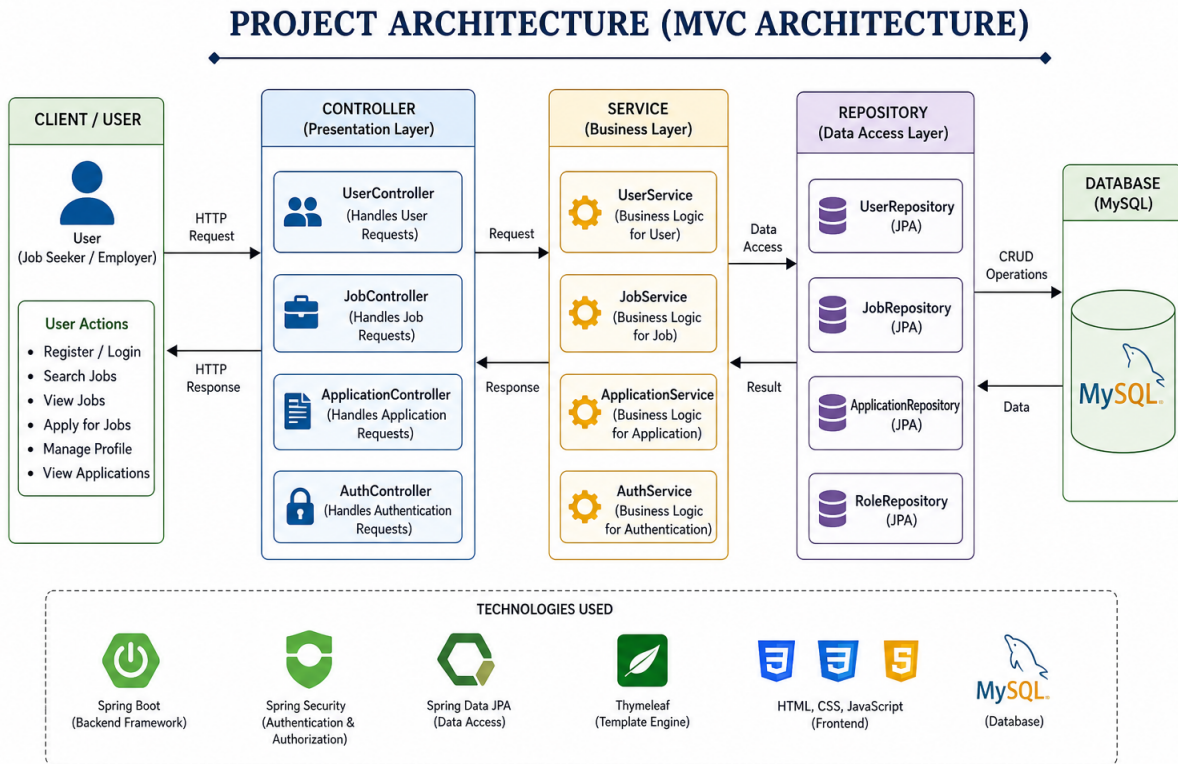


Figure 4.1: MVC Architecture of Job Portal System

Figure 4.1 shows The MVC architecture of the Job Portal system divides the application into three main components: Model, View, and Controller. The View layer, implemented using Thymeleaf templates, is responsible for presenting the user interface and displaying job-related information to users. The Controller layer, built with Spring Boot, handles user requests, processes business logic, and coordinates communication between the frontend and backend. The Model layer interacts with the database through JPA repositories, ensuring efficient storage and retrieval of job, user, and application data. This structured separation improves maintainability, scalability, and overall system performance.

## 4.2 DataFlow Diagram

The Data Flow Diagram (DFD) represents how data moves through the system. It shows the interaction between users, the application, and the database.

- Users provide input through the frontend (login, registration, job search).
- The controller processes the request and sends it to the service layer.
- The service layer interacts with the repository to access the database.
- The database returns the required data.
- The result is displayed to the user through the frontend.

The data movement within the system is illustrated in [Figure4.2], showing how information flows between different components during execution.

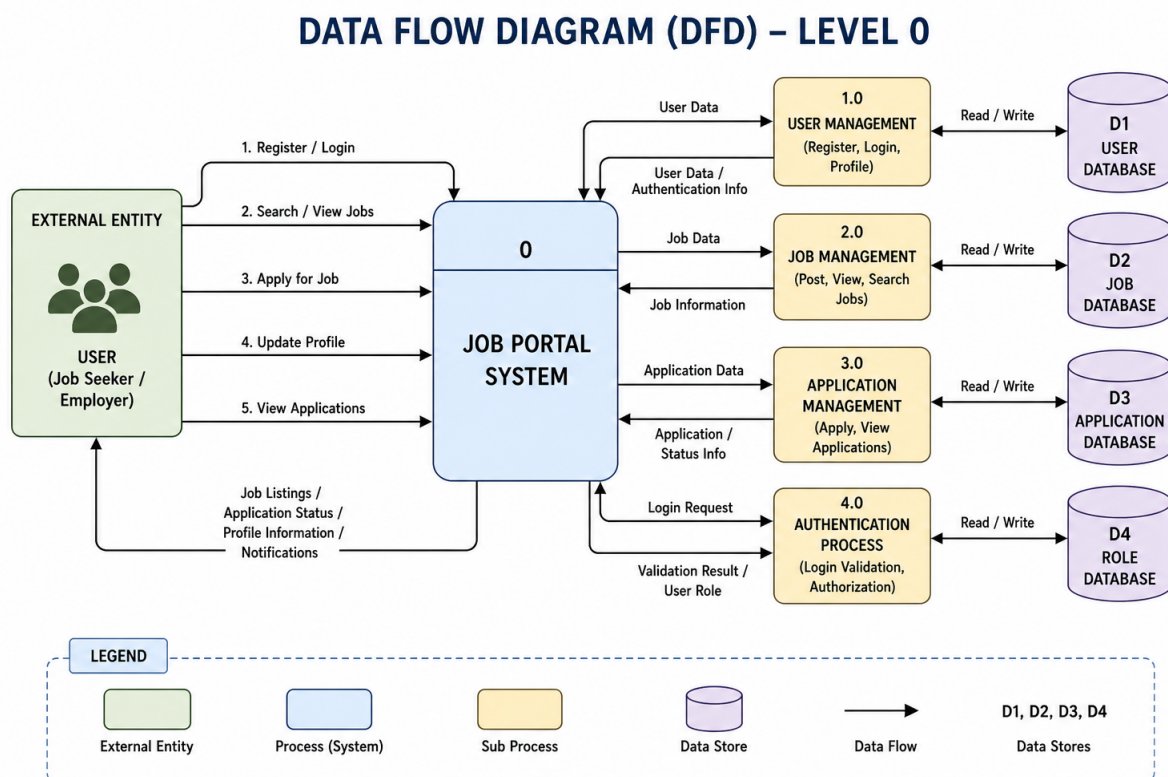


Figure 4.2: Data Flow Diagram

Figure 4.2 shows the Data Flow Diagram (DFD) Level 0 of the Job Portal System, illustrating the overall flow of data between users and the system. The external entity, which includes job seekers and employers, interacts with the system by performing actions such as registration, login, job search, job application, and profile management. The system processes these inputs through various modules including user management, job management, application management, and authentication. Each module communicates with corresponding data stores such as user database, job database, application database, and role database to store and retrieve relevant information. The diagram clearly represents how data flows between processes and storage components, ensuring efficient handling of user requests and maintaining data integrity within the system.

### **4.3 Module 1: User Management**

This module handles user-related functionalities such as registration, login, and profile management.

- User registration with basic details
- Secure login authentication
- Profile viewing and updating

### **4.4 Module 2: Job Management**

This module is responsible for managing job postings.

- Employers can post job vacancies
- Job details such as title, description, and company information are stored
- Users can browse available jobs

## 4.5 Module 3: Application Management

This module allows users to apply for jobs and manage applications.

- Users can apply for jobs
- Applications are stored in the database
- Employers can view applicants

### Advantages of this Project

The proposed system provides several advantages:

- a) User-Friendly Interface:** The application provides a simple and easy-to-use interface for both job seekers and employers.
- b) Efficient Data Management:** All data is stored and managed using MySQL, ensuring quick access and reliability.
- c) Scalable Architecture:** The use of Spring Boot and MVC architecture allows the system to be easily extended with additional features.

### Disadvantages

- Limited advanced features such as AI-based job recommendations.
- No email notification system for job updates.
- Requires internet connection for access.

## Chapter 5

# RESULTS AND DISCUSSIONS

The developed Job Portal Application was successfully implemented using Spring Boot, Thymeleaf, and MySQL. The system was tested for various functionalities including user registration, login, job posting, and job application[table 5.1].

The application provides a smooth and user-friendly interface, allowing users to easily interact with the system. The backend processes requests efficiently, and the database ensures secure and reliable data storage.

### **Observed Results:**

- Users are able to register and log in successfully.
- Employers can post job vacancies without errors.
- Job seekers can search and apply for jobs.
- Data is stored and retrieved correctly from MySQL database.
- The system performs efficiently with minimal response time.

| <b>Features</b> | <b>Existing Systems</b> | <b>Proposed System</b>   |
|-----------------|-------------------------|--------------------------|
| User Interface  | Complex                 | Simple and User-Friendly |
| Cost            | Paid Features           | Free to Use              |
| Job Search      | Available               | Available                |
| Job Application | Available               | Available                |
| Customization   | Limited                 | Flexible                 |
| Performance     | Moderate                | Efficient                |

Table 5.1: Comparison of Existing Systems and Proposed System

Table 5.1 shows the comparison between existing systems and the proposed job portal system based on various features. The existing systems often have complex user interfaces and require paid features, whereas the proposed system provides a simple and user-friendly interface with free access. Both systems support job search and job application functionalities; however, the proposed system offers better customization and flexibility. Additionally, the performance of the proposed system is more efficient compared to the moderate performance of existing systems. This comparison highlights the improvements and advantages of the proposed system over traditional platforms.

## 5.1 System Screenshots and Results

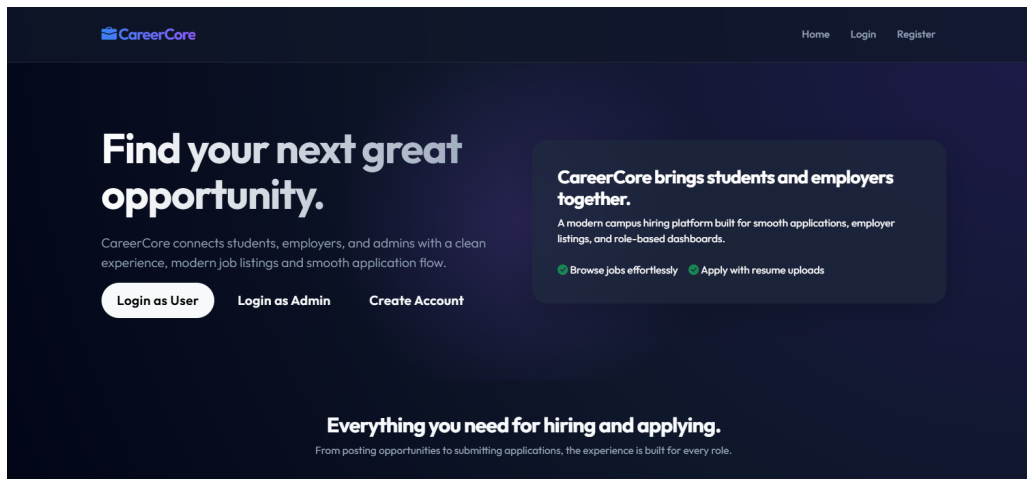


Figure 5.1: Login Output Screen

Figure 5.1 shows The login page allows users to securely access the job portal system. Users can enter their credentials to authenticate into the platform. The system validates inputs and prevents unauthorized access. This page acts as the entry point for all users.

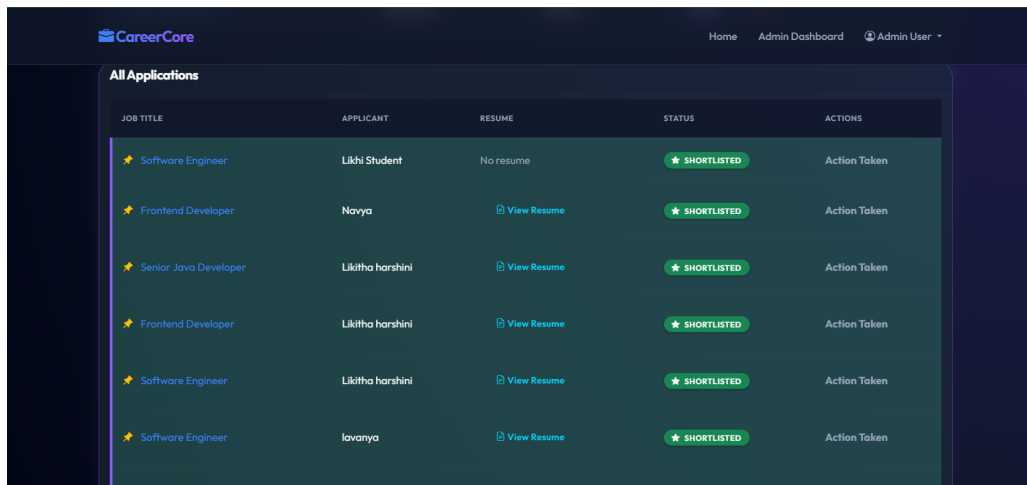
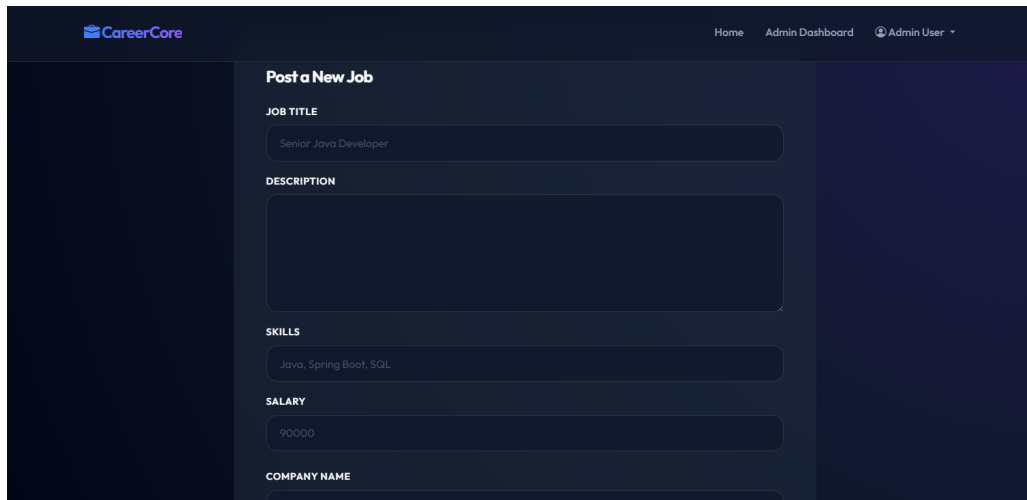


Figure 5.2: Employer Dashboard Output Screen

Figure 5.2 shows The employer dashboard displays job postings and applicant details. Employers can manage job listings efficiently from this interface. It allows

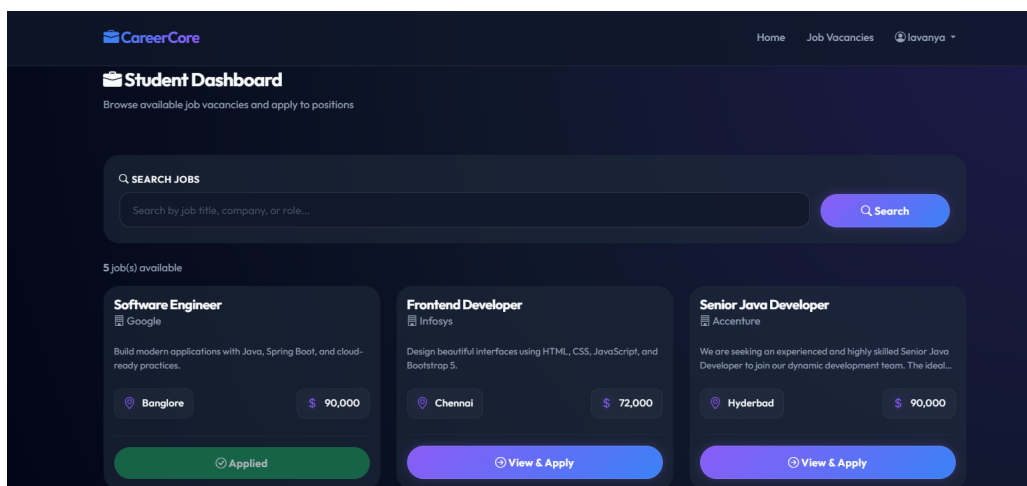
adding, editing, and deleting job postings. This improves recruitment management and decision-making.



The screenshot shows the 'Post a New Job' form in the CareerCore system. The form is titled 'Post a New Job' and is located in the 'Admin Dashboard' under the 'Admin User' profile. The form contains several input fields: 'JOB TITLE' (with the value 'Senior Java Developer'), 'DESCRIPTION' (a large text area), 'SKILLS' (with the value 'Java, Spring Boot, SQL'), 'SALARY' (with the value '90000'), and 'COMPANY NAME' (empty). The form is styled with a dark blue background and white text.

Figure 5.3: Edit Job Output Screen

Figure 5.3 shows The edit job page enables modification of job details. Users can update job title, description, and requirements. It ensures job listings remain accurate and updated. This feature provides flexibility to administrators.



The screenshot shows the 'Student Dashboard' in the CareerCore system. The dashboard is titled 'Student Dashboard' and is located in the 'Job Vacancies' section under the 'Javanya' profile. It features a search bar with the text 'SEARCH JOBS' and a search button. Below the search bar, it displays '5 job(s) available'. The jobs are listed in three columns: 'Software Engineer' at Google in Bangalore for \$90,000, 'Frontend Developer' at Infosys in Chennai for \$72,000, and 'Senior Java Developer' at Accenture in Hyderabad for \$90,000. Each job listing includes a brief description and a 'View & Apply' button.

Figure 5.4: Student Dashboard Output Screen

Figure 5.4 shows The student dashboard shows available job opportunities. Users can browse and apply for jobs easily. It helps track application status efficiently. This improves the user experience for job seekers.

Figure 5.5: Student Profile Output Screen

Figure 5.5 shows The student profile page stores user personal information. Users can upload resumes and update their skills. It helps recruiters evaluate candidates effectively. This ensures better job matching and selection.

## **Chapter 6**

# **CONCLUSION AND FUTURE ENHANCEMENTS**

### **6.1 Conclusion**

The Job Portal Application was successfully designed and implemented using Spring Boot, Thymeleaf, and MySQL. The system provides an efficient platform for job seekers and employers to interact, reducing the complexity of traditional recruitment processes.

The application supports essential functionalities such as user registration, login authentication, job posting, and job application. The use of MVC architecture ensures a well-structured and maintainable system. The integration of frontend and backend technologies enables smooth data flow and dynamic content rendering.

Overall, the project meets its objectives by providing a simple, scalable, and user-friendly solution. It demonstrates the effective use of modern web technologies in building real-world applications.

## 6.2 Future Enhancements

Although the system performs well, several improvements can be made in the future to enhance its functionality:

- **Email Notifications:** Notify users about job updates and application status.
- **Resume Upload Feature:** Allow users to upload and manage resumes.
- **Advanced Search Filters:** Improve job search with filters like salary, location, and experience.
- **Admin Dashboard:** Add admin control for managing users and job postings.
- **Mobile Application:** Develop an Android/iOS version of the system.
- **AI-based Recommendations:** Suggest jobs based on user skills and preferences.

## 6.3 Existing Applications and Review

- **LinkedIn**

<https://www.linkedin.com>

LinkedIn is a professional networking platform that allows users to build profiles, connect with professionals, and apply for jobs. It provides job recommendations based on user profiles and skills. However, it can be overwhelming for freshers due to its networking-heavy approach and competitive environment.

- **Naukri**

<https://www.naukri.com/>

Naukri is one of the most popular job portals in India, offering a wide range of job listings across various industries. It provides resume building, job alerts,

and recruiter connections. However, the platform often contains outdated job postings and requires premium services for better visibility.

- **Indeed**

`https://www.indeed.com/`

Indeed is a global job search platform that aggregates listings from multiple sources, making it easy for users to find opportunities. It offers company reviews, salary insights, and a simple application process. However, it lacks personalized filtering and may display duplicate or irrelevant job listings.

### **6.3.1 Application Access**

The developed Event Booking System is currently hosted in a local environment (localhost) for testing and demonstration purposes. The application consists of the following modules, accessed in a sequential workflow:

- **User Registration Page**

URL: `http://localhost:8080/register`

Allows new users to create an account by entering their personal and login details.

- **User Login Page**

URL: `http://localhost:8080/login`

Enables registered users to securely log into the system using their credentials.

- **Job Portal Page**

URL: `http://localhost:8080/`

Displays available job listings and allows users to browse and apply for jobs.

- **User Application Page**

URL: `http://localhost:8080/jobs/2/apply`

Allows users to submit job applications by providing required details and resume.

- **Admin Dashboard**

URL: `http://localhost:8080/admin/dashboard`

Provides an overview of system activities including jobs, users, and applications.

- **Post New Job Page**

URL: `http://localhost:8080/jobs/new`

Allows administrators to create and publish new job postings.

- **Edit Job Page**

URL: `http://localhost:8080/jobs/1/edit`

Enables administrators to modify existing job details.

- **View Resume Page**

URL: `http://localhost:8080/admin/applications/4/resume`

Allows administrators to view resumes submitted by applicants.

**Note:** These URLs are accessible only within the local system environment and are used for development and testing purposes.

## 6.4 SOURCE CODE IMPLEMENTATION

### Login Page

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
<meta charset="UTF-8" />
<title>Login - Job Portal</title>
```

```

<link rel="stylesheet" th:href="@{/css/style.css}" />
</head>
<body>
<div class="container page-grid">
<header>
<div>
<h1>Job Portal </h1>
<p>Login to manage job applications , post roles , or browse jobs.</p>
</div>
<nav>
<a th:href="@{/}">Home</a>
<a th:href="@{/register}">Register </a>
</nav>
</header>

<section class="hero">
<div>
<h1>Welcome back </h1>
<p>Sign in to continue your journey.</p>
</div>
</section>

<section class="auth-form">
<h1>Sign In </h1>
<form method="post" th:action="@{/login}">
<label>Username </label>
<input type="text" name="username" required />

<label>Password </label>
<input type="password" name="password" required />

<button type="submit">Sign In </button>
</form>

<p>D o n t have an account?
<a th:href="@{/register}">Register </a></p>
<div th:if="${param.error}">Invalid username or password</div>
<div th:if="${param.logout}">Logged out successfully </div>
<div th:if="${param.registered}">Registration successful </div>

```

```
</section>
</div>
</body>
</html>
```

## Student Dashboard

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
<meta charset="UTF-8" />
<title>Student Dashboard</title>
<link rel="stylesheet" th:href="@{/css/style.css}" />
</head>
<body>

<div class="container page-grid">
<header>
<h1>Student Dashboard</h1>

<nav>
<a th:href="@{/}">Browse Jobs</a>
<a th:href="@{/applications}">My Applications</a>
<a th:href="@{/profile}">Profile</a>
<a th:href="@{/logout}">Logout</a>
</nav>
</header>

<section>
<h2>Available Jobs</h2>

<div th:each="job : ${jobs}">
<h3 th:text="${job.title}"></h3>
<p th:text="${job.description}"></p>

<p><strong>Location:</strong>
<span th:text="${job.location}"></span></p>
```

```

<p><strong>Employer:</strong>
<span th:text="${job.employer.fullName}"></span></p>

<form th:action="@{/jobs/' + ${job.id} + '/apply'}" method="post">
<button type="submit">Apply Now</button>
</form>

</div>
</section>

</div>
</body>
</html>

```

## Application Tracking Page

```

<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
<meta charset="UTF-8" />
<title>My Applications</title>
<link rel="stylesheet" th:href="@{/css/style.css}" />
</head>
<body>

<div class="container page-grid">

<header>
<h1>My Applications</h1>

<nav>
<a th:href="@{/dashboard}">Dashboard</a>
<a th:href="@{/profile}">Profile</a>
<a th:href="@{/logout}">Logout</a>
</nav>
</header>

<section>

```

```

<div th:if="${applications.isEmpty()}">
No applications yet.
</div>

<div th:each="app: ${applications}">
<h3 th:text="${app.job.title}"></h3>

<p><strong>Employer:</strong>
<span th:text="${app.job.employer.fullName}"></span></p>

<p><strong>Status:</strong>
<span th:text="${app.status}"></span></p>

<p><strong>Applied On:</strong>
<span th:text="${app.appliedAt}"></span></p>

</div>

</section>
</div>

</body>
</html>

```

# Chapter 7

## ADDRESSING COMPLEX ENGINEERING PROBLEM AND SDG

### 7.1 Mapping of the Project to Complex Engineering Problem (CEP)

| Level | Attribute                | Characteristics                         | Wks           | Justification                                                      |
|-------|--------------------------|-----------------------------------------|---------------|--------------------------------------------------------------------|
| WP1   | Depth of Knowledge       | Requires in-depth engineering           | WK3, WK4, WK5 | Requires knowledge of Java, Spring Boot, JPA, and database design. |
| WP2   | Conflicting Requirements | Technical and non-technical constraints | WK4, WK5      | Balances usability and security.                                   |
| WP3   | Depth of Analysis        | Analytical and design thinking          | WK3, WK4      | Includes ER design, APIs, resume upload.                           |
| WP4   | Familiarity              | Partially new problems                  | WK4           | Job tracking beyond CRUD systems.                                  |
| WP5   | Applicable Codes         | Limited standard solutions              | WK6           | Custom logic for resume and filtering.                             |
| WP6   | Stakeholder Needs        | Multiple stakeholders                   | WK5, WK6      | Supports job seekers, recruiters, admin.                           |
| WP7   | Interdependence          | System integration                      | WK3, WK5      | Integrates UI, backend, DB, validation.                            |

Table 7.1: Mapping of CEP Attributes

## 7.2 Mapping of the Project to Complex Engineering Activities (CEA)

| EA No. | Attribute             | Characteristics            | Justification                                   |
|--------|-----------------------|----------------------------|-------------------------------------------------|
| EA1    | Range of Resources    | Use of technologies        | Java, Spring Boot, MySQL, HTML, CSS, JavaScript |
| EA2    | Level of Interactions | Complex module interaction | Handles jobs, applications, login, and uploads  |
| EA3    | Innovation            | Modern technologies        | Role-based dashboards, filtering, email alerts  |
| EA4    | Consequences          | Impact on users            | Improves hiring and job accessibility           |
| EA5    | Familiarity           | Beyond standard            | Combines tracking, resume, and admin system     |

Table 7.2: Mapping of CEA Attributes

## 7.3 SDG Mapping

| SDG No. | SDG Title           | Relevance                                         |
|---------|---------------------|---------------------------------------------------|
| SDG 8   | Decent Work         | Connects job seekers and employers for employment |
| SDG 9   | Innovation          | Uses modern web technologies for recruitment      |
| SDG 10  | Reduced Inequality  | Equal job access for all users                    |
| SDG 16  | Strong Institutions | Ensures security using authentication system      |

Table 7.3: SDG Mapping

# References

- [1] LinkedIn Job Portal. A professional networking platform that allows users to create profiles, connect with professionals, and apply for jobs. Available at:  
<https://www.linkedin.com>
- [2] Naukri Job Portal. A leading Indian job portal that enables users to upload resumes, search for jobs, and receive recruiter notifications. Available at:  
<https://www.naukri.com>
- [3] Indeed Job Portal. A global job search engine that aggregates job listings from multiple sources and allows easy application. Available at:  
<https://www.indeed.com>
- [4] Glassdoor Job Portal. Provides job listings along with company reviews, salary insights, and interview experiences shared by employees. Available at:  
<https://www.glassdoor.com>
- [5] Monster Job Portal. An online employment platform offering job listings, resume services, and career advice tools. Available at:  
<https://www.monster.com>
- [6] Shine Job Portal. An Indian job portal that provides job listings, career guidance, and resume building services. Available at: <https://www.shine.com>
- [7] Spring Boot Documentation. Official documentation providing guidance for building production-ready applications using Spring Boot framework.

Available at: <https://spring.io/projects/spring-boot>

- [8] Spring Framework Documentation. Comprehensive documentation for developing Java applications using the Spring Framework. Available at: <https://spring.io/projects/spring-framework>
- [9] Thymeleaf Documentation. A modern server-side Java template engine used for building dynamic web applications. Available at: <https://www.thymeleaf.org>
- [10] MySQL Documentation. Official reference for MySQL database, covering installation, configuration, and query management. Available at: <https://dev.mysql.com/doc/>
- [11] Oracle Java Documentation. Provides detailed information about Java programming, APIs, and development tools. Available at: <https://docs.oracle.com/en/java/>
- [12] W3Schools HTML Tutorial. Beginner-friendly tutorial covering HTML basics and web page structure. Available at: <https://www.w3schools.com/html/>
- [13] W3Schools CSS Tutorial. Provides knowledge about styling web pages using Cascading Style Sheets. Available at: <https://www.w3schools.com/css/>
- [14] W3Schools JavaScript Tutorial. Explains JavaScript fundamentals for creating interactive web pages. Available at: <https://www.w3schools.com/js/>
- [15] GeeksforGeeks Spring Boot Guide. Educational resource explaining concepts, examples, and implementation of Spring Boot. Available at: <https://www.geeksforgeeks.org/spring-boot/>